



Learn to Refine: Synergistic Multi-Agent Path Optimization for Lifelong Conflict-Free Navigation of Autonomous Vehicles

Junjun Li

South China University of Technology
Guangzhou, China
blueslj9@gmail.com

Ting Huang*

Xidian University
Guangzhou, China
gnauhgnith@gmail.com

Zeyuan Ma

South China University of Technology
Guangzhou, China
scut.crazynicolas@gmail.com

Yue-Jiao Gong*

South China University of Technology
Guangzhou, China
gongyuejiao@gmail.com

Abstract

Lifelong Multi-Agent Path Finding (LMAPF) focuses on planning conflict-free paths for agents, like autonomous vehicles, that are continuously assigned new tasks. The synergy of search-based and learning-based methods holds promise for striking a balance in-between effectiveness and efficiency but still faces several challenges such as inferior initial paths, weak search-learning synergy and low sample utilization rate. To address these issues, this paper proposes a new synergized LMAPF approach, named Synergistic Multi-Agent Path Optimization (SMAPO), which consists of two tightly-coupled phases: Primordial Planning and Decision Refinement. In the Primordial Planning phase, we introduce a novel load-balanced A* algorithm that integrates planned and perceived congestion costs, which enhances initial solution quality by evenly distributing spatiotemporal traffic loads, thereby mitigating potential conflicts. In the Decision Refinement phase, we propose a novel Encoder-Decoder based neural network to learn a collaborative optimization policy through multi-agent reinforcement learning. In addition, we leverage dual transformations to augment trajectory samples during online learning, enhancing both the sample utilization rate and overall learning stability. Extensive experiments reveal that our SMAPO is superior to the state-of-the-art baselines in effectiveness, efficiency, and generalization capability. Source code is available at <https://github.com/ByteUser-blues/SMAPO>.

CCS Concepts

• **Computing methodologies** → **Multi-agent reinforcement learning; Multi-agent planning; Heuristic function construction;**
• **Applied computing** → **Transportation;** • **Computer systems organization** → **Robotic autonomy.**

*T. Huang and Y.-J. Gong are the corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

KDD '25, Toronto, ON, Canada

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1454-2/2025/08
<https://doi.org/10.1145/3711896.3737017>

Keywords

Lifelong multi-agent path finding, Multi-agent reinforcement learning, Data-driven optimization, Autonomous vehicles

ACM Reference Format:

Junjun Li, Zeyuan Ma, Ting Huang, and Yue-Jiao Gong. 2025. Learn to Refine: Synergistic Multi-Agent Path Optimization for Lifelong Conflict-Free Navigation of Autonomous Vehicles. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25)*, August 3–7, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3711896.3737017>

1 Introduction

Multi-agent path finding (MAPF) aims to find a set of cost-effective and conflict-free paths for autonomous vehicles, navigating them to reach their goal positions [37]. Lifelong MAPF (LMAPF) considers more practical challenges by continually planning the agents' paths to new goal positions throughout their lifetime [19]. LMAPF has extensive application scenarios, including warehouse logistics [24], traffic management [14], automated valet parking [28] and airport operations [27]. The challenge of LMAPF over MAPF is not only resolving immediate conflicts but also ensuring lifelong efficiency (e.g., throughput). The prevalent MAPF baseline method is search-based, represented by conflict-based search (CBS) [32], which individually plans each agent's path and exhaustively resolves conflicts of the path set with the lowest cost. Subsequent MAPF works employ suboptimal CBS-like methods by relaxing cost bounds [18, 39] or prioritizing primary conflicts [23, 39] to more quickly search for feasible and cost-effective paths. To extend search-based methods to LMAPF, current work decomposes LMAPF into a series of MAPF instances with paths for all agents replanned at each time step [9] or within a time window [19, 38]. However, as scenario complexity increases (e.g., higher agent and obstacle density), planning times grow significantly, and overall performance deteriorates. This challenge becomes more severe in LMAPF, where frequent replanning further increases computational overhead.

Recently, learning-based methods have emerged as a novel and effective paradigm for both optimization [3, 10, 25] and planning [4, 8, 15]. In planning task, neural networks are utilized to learn the planning priority order of agents [42] or generate feasible paths in an end-to-end manner [30]. Thus, it allows direct extension from MAPF to LMAPF while avoiding the extensive online computation required by search-based methods. By facilitating effective

multi-agent reinforcement learning (MARL) and imitation learning (IL), learning-based methods train agents to search feasible paths through efficient collaboration [1, 7, 22, 26, 40]. These methods promise scalable and efficient decision making. However, they encounter challenges, like the data-driven learning fashion poses challenges in sample utilization rate and overall learning effectiveness, often resulting in poor generalization to unseen maps, which are less generic and lower throughput than search-based methods.

To combine the respective strengths of search and learning methods, recent approaches have explored the integration of search-based and learning-based methods. MATS-LP algorithm [35] incorporates Monte Carlo tree search to enhance the policy network. Switcher method [36] investigates an adaptive switching strategy between search and learning methods. Furthermore, Follower method [34] adopts a more integrated approach by integrating heuristic search for path planning and reinforcement learning for conflict resolution. Despite their potential, these hybrid approaches face critical limitations: their search components either generate myopic initial paths (Follower, Switcher) or demand excessive computation (MATS-LP), while weak search-learning synergy undermines both sample efficiency and generalization.

To address these challenges, we introduce SMAPO (Synergistic Multi-Agent Path Optimization), a novel search-integrated learning-based approach that enables rapid generation and optimization of load-balanced path sets across diverse scenarios using a single unified model. SMAPO introduces three key contributions:

- 1) SMAPO integrates search and learning in a tightly coupled, two-phase framework. In the Primordial Planning phase, a specifically designed A* algorithm generates initial paths, which are then refined using MARL in the Decision Refinement phase. The MARL-based planning results feed real-time adjustments to the congestion costs back into the A* planning. This feedback loop dynamically adapts to environment changes, improving both short-term and long-term path optimization.

- 2) In the Primordial Planning phase, we introduce a novel load-balanced A* algorithm that integrates both planned and perceived congestion costs into the agents' spatio-temporal paths. This search-based approach reduces conflicts while maintaining efficiency by proactively balancing traffic loads across agents. The algorithm enhances the quality of the initial path set, which serves as the foundation for the subsequent learning phase.

- 3) In the Decision Refinement phase, we propose an Encoder-Decoder-based architecture to learn a collaborative optimization policy through multi-agent reinforcement learning. To stabilize MARL training, we propose a sample-efficient Dual Transformation strategy that augments trajectories with dual variants, enhancing generalization and convergence.

In summary, SMAPO enhances the synergy between search and learning mechanisms, significantly improving generalization capabilities across diverse scenarios while maintaining computational efficiency in generating conflict-free and cost-effective path set. Experimental results demonstrate that SMAPO significantly outperforms existing learning-based methods while achieving a superior throughput-efficiency balance compared to search-based approaches. Further ablation studies demonstrate core designs of SMAPO collaboratively contribute to superior performance.

2 Related Works

2.1 Multi-Agent Path Finding

Methods for solving MAPF problem can be divided into two categories: search-based methods and learning-based methods. The essence of search-based methods for solving MAPF problems is the use of search techniques to conduct centralized path planning and conflict resolution, with the most notable being CBS [32], a complete and optimal two-level solver. The core of CBS lies in starting from a set of shortest but infeasible paths and exhaustively resolving the existing conflicts. Subsequently, CBS variants, such as CBICS [39] and EECBS [18], accelerate the CBS algorithm with heuristic search for conflict selection and resolution. Another variant, PBS [23], combines the ideas of Cooperative A* (CA*) [33], where the high-level planning is similar to CBS, but the conflict resolution involves assigning higher priority to one of the conflicting agents instead of adding additional constraints.

Compared to search-based methods, learning-based methods leverage neural network parallelism to enhance algorithmic efficiency, enabling stable scalability to scenarios with larger agent counts. This has led to a wealth of influential work, such as PRIMAL [30], combined IL and reinforcement learning to solve the MAPF problem. Several studies have primarily focused on achieving global information exchange among agents through communication, thereby improving the quality of the solutions, including MAGAT [21], DHC [26], and SCRIMP [40]. The latest work MAPF-GPT [1] utilizes the powerful fitting capability of DiffuserLites to replicate expert behaviors, showcasing its potential in capturing and generalizing complex decision-making patterns.

2.2 Lifelong Multi-Agent Path-Finding

In LMAPF, the long-term nature of continuous goal allocation imposes greater demands on the algorithm's real-time performance and adaptability, making search-based MAPF methods difficult to apply directly. An intuitive approach is to decompose LMAPF into a series of MAPF instances, replanning paths for all agents at each time step [9], which leads to considerable planning time. The state-of-the-art method for leveraging the advanced centralized MAPF solvers to address LMAPF tasks is RHCR [19], which utilizes cost-bounded MAPF solvers to solve LMAPF problems, balancing the path cost minimization and conflict resolution efficiency. Compared to the existing works, our study proactively balances traffic loads across agents, improving scalability to higher agent and obstacle densities while enhancing overall efficiency.

Unlike search-based MAPF methods, learning-based MAPF approaches utilize neural networks for single-step decision-making, enabling direct extension to LMAPF problems. Notable examples include the extended version of PRIMAL, PRIMAL2 [7], which enhances agent collaboration and environmental understanding through detailed feature map design, as well as advanced learning-based MAPF methods like SCRIMP, MAPF-GPT, all of which can be directly applied to LMAPF. Recently, several algorithms have integrated search-based strategies to enhance learning-based methods. For instance, MATS-LP [35] improves the learned policy by incorporating Monte Carlo Tree Search (MCTS) while Follower [34] synergizes A* with learning-based techniques to enhance both training efficiency and effectiveness. The synergy of search-based and

learning-based methods holds promise for balancing effectiveness and efficiency. However, the loosely coupled frameworks of existing methods still lead to issues such as myopic path generation and substantial computation overhead. To address these issues, SMAPO introduces several novel design innovations that enhance search-learning synergy, enabling more cost-efficient path optimization.

3 Preliminaries

3.1 Problem Formulation

Given a graph $G = (V, E)$, where V corresponds to locations and E represents connections between adjacent locations, and a set of n agents $\mathcal{N}$ with corresponding initial start position set ST and goal position set ED . The agents request new tasks from a task allocator, which operates externally to our path-planning system, whenever they reach their current goals. Time is discretized into steps, and in each time step, each agent can move to an adjacent location or wait in its current position (actions). Conflicts occur when two agents occupy the same location simultaneously (i.e., vertex conflict) or traverse the same edge in opposite directions at the same time step (i.e., edge conflict). The optimization objective of LMAPF is to continuously plan a set of conflict-free paths for agents, and maximize *throughput* (the average number of goals of ED reached by all agents per time step).

3.2 POMDP and MARL

Let $M = \langle \mathcal{N}, \mathcal{S}, \mathcal{A}, P, R, O, \gamma \rangle$ be the multi-agent extension of Partially Observable Markov Decision Process (POMDP) [2], where $\mathcal{N} = \{1, \dots, n\}$ is the set of agents, $\mathcal{S}$ is the joint state space, $\mathcal{A}$ is the joint action space. The transition function $P : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ (where $\Delta(\cdot)$ is the probability simplex) is determined by the environment, the reward function $R : \mathcal{S} \times \mathcal{A} \rightarrow [0, R_{max}]$ defines the joint reward, $\mathcal{O}$ denotes the joint observation space, and the discount factor $\gamma \in [0, 1)$ governs the importance of future rewards. $\mathcal{S}$, $\mathcal{A}$ and $\mathcal{O}$ can be decomposed into: $S_1 \times \dots \times S_n$, $A_1 \times \dots \times A_n$ and $O_1 \times \dots \times O_n$, where S_k , A_k and O_k for $k \in [1, n]$ represents the individual state, action and observation space of each agent.

In this paper, we write $i_{1:m}$ to denote the set of all agents including $i_1, \dots, i_m$, $\pi_{i_{1:m}} = \pi_{i_1} \times \dots \times \pi_{i_m}$, the same applies to $o_{i_{1:m}}, s_{i_{1:m}}, a_{i_{1:m}}$. The notation $-i$ represents all agents other than i . A joint policy $\pi_{1:n} : \mathcal{S} \rightarrow \mathcal{A}$ describes a distribution over joint action for each joint state, which can be decomposed into: $\pi_1 \times \dots \times \pi_n$. Since in LMAPF all agents advance in a collaborative way, the global one-step reward is hence the sum of local one-step rewards from each agent $R_t(s_{1:n}^t, a_{1:n}^t) = \sum_{i=1}^n r_i(s_i^t, a_i^t | a_{-i}^t)$. The objective of MARL is to learn the optimal joint policy $\pi_{1:n}^*$ that maximizes the cumulative collective reward $\sum_t E_{a_{1:n}^t \sim \pi_{1:n}^*} [\gamma^t R(s_{1:n}^t, a_{1:n}^t)]$. We introduce the detailed MARL designs of SMAPO in Section 4.2.

4 Methodology

As shown in Figure 1, our synergistic multi-agent path optimization algorithm, named **SMAPO**, consists of two phases. The *Primordial Planning phase* utilizes Load-Balanced A* to quickly generate an initial paths $\mathbb{S}$ that balance the map's traffic load. The *Decision Refinement phase* then further refine the planing results to avoid potential conflicts. These two phases would engage the overall

planning process in turn to provide cost-efficient and conflict-free paths online. We elaborate this collaboration paradigm next.

4.1 Primordial Planning (PP) Phase

The purpose of Primordial Planning phase is to obtain an initial set of paths through fast-search algorithm. To achieve this, we propose the Load-Balanced A* (LB-A*) algorithm, illustrated in the purple section of Figure 1, which quickly generates initial paths of agents that balance the traffic load on the map. During the execution of LB-A*, the individual cost of a cell c at time step t (when transitioning into it) consists of two components: the Planned Congestion Cost (PICC) and the Perceived Congestion Cost (PeCC):

$$cost(c, t) = cost_{PICC}(c, t) + cost_{PeCC}(c, t). \quad (1)$$

Once we obtain the costs, we could employ an analog of traditional A* algorithm to sequentially plan paths for agents in LMAPF.

4.1.1 Planned Congestion Cost. Motivation arises when generating paths with A*, where conflict occurrences follow the principle that: if one agent traverses a cell at time t_1 and another agent traverses the same cell at time t_2 , the likelihood of a conflict occurrence decreases as $|t_2 - t_1|$ increases. This observation leads to the development of PICC, a novel mechanism designed to partition the temporal domain into multiple segments while meticulously accounting for the trajectories of moving agents. Concretely, to estimate the PICC of the cell (x, y) at time t , we first, for the previously planned trajectories, count the number of agents located in the grid cell (x, y) at the time step $t + k$, and denote the count as $num_k = Count(x, y, t + k)$, where $k \in [-\Delta t, \Delta t]$ and the function *Count* counts the number of occurrences of a given triple in the set *Planned Paths*. The cost of time $t_0 + k$ for the grid cell (x_0, y_0) is calculated as:

$$f(num_k) = \begin{cases} \alpha * num_k^2, & k = 0, \\ \beta * num_k * \lambda^{|k|}, & k \neq 0, \end{cases} \quad (2)$$

where $\lambda \in (0, 1]$ controls the decay of the cost impact as function of the temporal distance. α and β are parameters employed to modulate the cost base when agents occupy the grid cell at the same time ($k = 0$) and different times ($k \neq 0$). In the training and testing conducted in paper, we set $\alpha = 2$, $\beta = 0.5$, $\lambda = 0.8$.

The PICC for the grid cell (x, y) at time t is the sum of the cost contributions from all time points within the interval $[t - \Delta t, t + \Delta t]$, given by:

$$cost_{PICC}(x, y, t) = 1 + \sum_{k \in [-\Delta t, \Delta t]} f(Count(x, y, t + k)). \quad (3)$$

By adjusting these parameters, we can control the trade-off between path conflict avoidance and path length: By setting $\Delta t = 0$ and $\alpha = \infty$ in the computation of $f(num_k)$, we can obtain the optimal conflict-avoiding path, but this will substantially increase the time required to acquire the path. Conversely, by setting $\Delta t = 0$ and $\alpha = 0$, we can obtain the naive shortest path. When Δt is non-zero, PICC provides an interpolation between the shortest path (which may have conflicts) and the optimal conflict-free path, balancing both path length and conflict avoidance.

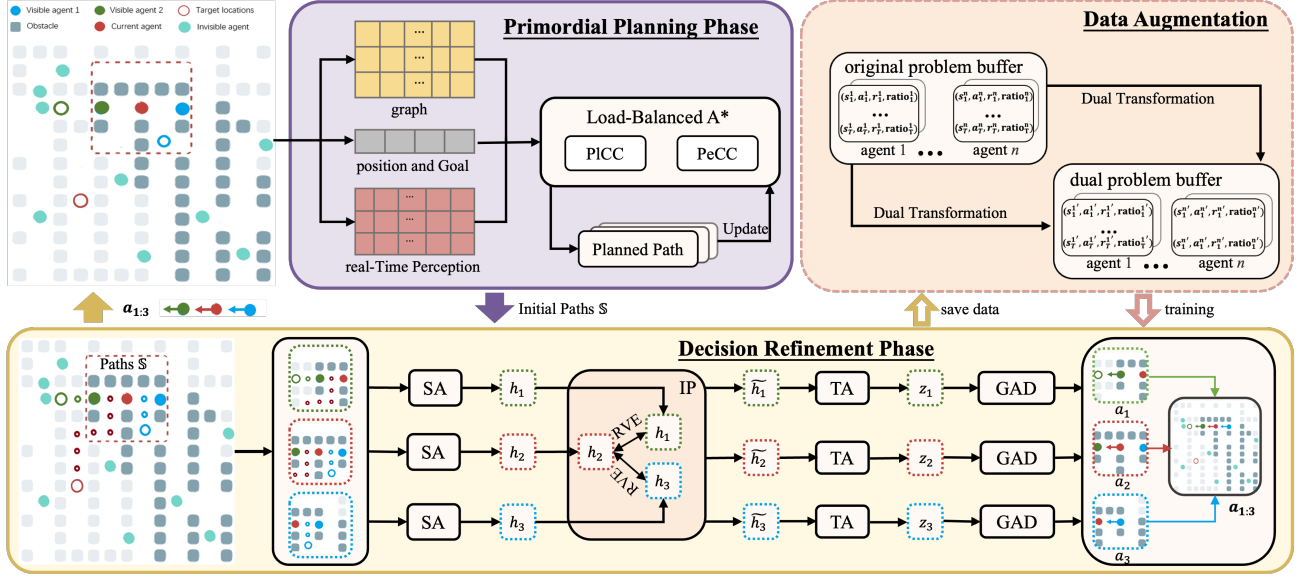


Figure 1: The general pipeline of the SMAPO approach. The action selection policy for each agent consists of two phases: the Primordial Planning phase (top-center) adopts Load-Balanced A* algorithm to address long-term path planning by balancing traffic loads and reducing potential conflicts, and the Decision Refinement phase (bottom) resolves immediate conflicts using neighborhood data. An additional Dual Transformation (top-right) enhances training data effectiveness to improve learning.

4.1.2 Perceived Congestion Cost. However, when utilizing the PICC alone, agents consider the impact of planned trajectories, but do not account for the potential influence of future unplanned agents. The avoidance of congestion between agents relies entirely on the planning of the subsequent agents, which places a heavy burden on the future agents. This can, to a certain extent, affect the congestion avoidance effectiveness between some agents, thereby impacting the overall efficiency of the multi-agent system. To address this, the Perceived Congestion Cost (PeCC) is proposed.

This PeCC is calculated based on the real-time field-of-view perception of each agent, allowing agents to account for their mutual influence, rather than just one agent influencing another. The cost design can relieve congestion during path search.

Furthermore, to ensure the timeliness of the PeCC, we introduce a decay factor as follows:

$$\text{cost}_{\text{PeCC}}(x, y, t) = \sum_{k \in [-t, 0]} \gamma^{t_0 - t} \text{AtCell}(x, y, t + k). \quad (4)$$

The indicator function $\text{AtCell}(x, y, t)$ returns 1 if an agent was observed (by the current agent) at the location (x, y) at timestep $t + k$, and 0 otherwise. Let c denote the cell at position (x, y) .

Leveraging the preceding results, we can demonstrate that the following recursive relationships are satisfied:

$$\text{cost}_{\text{PeCC}}(c, t_0 + 1) = \gamma \text{cost}_{\text{PeCC}}(c, t_0) + \text{AtCell}(c, t_0). \quad (5)$$

Now we summarize the procedure of the proposed LB-A*: for each agent in LMAPF, we sequentially employ LB-A* to obtain desired path for each agent. Since we embed PICC and PeCC into the cost function, we could easily achieve a collection of paths with balanced traffic load for all agents. We provide detailed procedure

of LB-A* in Algorithm 1, Appendix B. A informative state information O_t^i could be extracted from the paths collection and used for Decision Refinement phase, see detailed elaboration in Section 4.2.1.

4.2 Decision Refinement (DR) Phase

An intuitive insight on the planning results from PP phase is that: certain tradeoff exists in-between the load-balance and conflict-avoidance. This motivate us to further refine the unresolved conflicts from PP phase by MARL technique. Within this Decision Refinement (DR) phase, we tie each path derived from PP phase with a RL agent. These RL agents collaborate with each other to refine the paths according to their observations on the planning status. They are rewarded if the conflicts among all paths are relieved.

4.2.1 MDP Designs. State. Given that PP phase generates paths according to current and goal positions, we exclude encoding the agents' current and goal positions in the Decision Refinement phase. Instead, for agent i at timestep t , its observation $O_t^i := \{o_t^{i,1}, o_t^{i,2}\}$ includes obstacle information and the initial path from the PP phase ($o_t^{i,1}$), and the positions of nearby agents ($o_t^{i,2}$) where $o_t^{i,1}, o_t^{i,2} \in \mathbb{R}^{(2r+1) \times (2r+1)}$, r is the observation radius. **Action.** At time t , agent i selects one of five actions, corresponding to keeping still, moving up, moving down, moving left or moving right on the grid. **Reward.** At time t , when agent i reaches the adjacent point on the planned path, it receives a reward of $+r$ ($r = 0.01$ in this paper); otherwise, the reward is 0. **State Transition.** Once MARL refines all paths planned by PP phase, the refined version is returned to PP phase and go through a novel round of LB-A* to update planned paths. Then state O_{t+1}^i is extracted from the updated planning results.

4.2.2 Neural Network Designs. We develop an encoder-decoder structure, allowing agents to collaboratively decide their real-time actions based on observations.

Spatial Awareness (SA) module. We apply a ResNet [12] backbone combined with an additional multi-layer perceptron (MLP) layer to preprocess the observation O_t^i . This fusion process enables seamless extraction of spatially relevant feature information:

$$h_i^t = \phi(\text{RN}(O_t^i; \theta_{SA}); \mathbf{W}_{SA}), \quad (6)$$

where $\phi(\cdot; \mathbf{W}_{SA})$ denotes the MLP Layer with the shape of $d \times d$, RN denotes the ResNet backbone with 8 residual block.

Interaction Perception (IP) module. After extracting spatially relevant features h_i^t described above, the IP module enable inter-agent information exchange to support implicit priority allocation and collective decision-making, ultimately achieving high-quality, conflict-free path planning. Denote the set of all h_i^t at time step t as $\mathbf{H}^t$:

$$\mathbf{H}^t = \bigcup_{k=1}^n \{h_k^t\}. \quad (7)$$

By using id_i^t , agent i obtain the information set to be extracted from $\mathbf{H}^t$, denoted as $\bar{\mathbf{H}}_i^t$:

$$\bar{\mathbf{H}}_i^t = [h_{id_{i,1}}^t, \dots, h_{id_{i,m}}^t]^T \in \mathbb{R}^{m \times d}, \quad (8)$$

where m is the predefined maximum number of visible agents, if the number of visible agents is less than m , the remaining tensor is padded with zeros.

Since $|id_i^t|$ is of variable length, an intuitive approach to aggregating this information is to directly employ mask-attention mechanism. Moreover, inspired by [6, 16, 29] and considering the diversity of multi-agent collaboration patterns, we design **Relative Vector Embedding (RVE)** to address the permutation-invariant nature of Attention and the lack of positional information, effectively adapting to POMDP challenges in DR phase.

Specifically, the information to be aggregated in $\bar{\mathbf{H}}_i^t$ originates from agents within agent i 's field of view, implying that the relative position vector $(\Delta x, \Delta y)$ satisfies:

$$\max(\Delta x, \Delta y) \leq r, \quad \min(\Delta x, \Delta y) \geq -r. \quad (9)$$

Thus, the number of possible relative position vector is constrained to $(2r+1)^2$, here r is the observation radius. We employ a learnable embedding layer to represent all potential relative positions:

$$\mathbf{E}_{\theta_e} = \bigcup_{w=-r}^r \bigcup_{h=-r}^r \{e_{w,h}\}, \quad (10)$$

Similarly, we derive the corresponding relative vector embedding for agent i using id_i^t :

$$\bar{\mathbf{E}}_i^t = [e_{ve_1^i}, \dots, e_{ve_m^i}] \in \mathbb{R}^{m \times d}, \quad (11)$$

where ve_k^i denotes the relative vector of the k -th agent in id_i^t . By integrating relative tensor embeddings, the prioritization of different agent information is computed as follow:

$$p_{i,k}^t = \frac{(\mathbf{W}_Q h_i^t)^T (\mathbf{W}_K h_k^t) + b_{i,k}^t}{\sqrt{d}}. \quad (12)$$

Balancing performance and additional parameters, the computation of $b_{i,k}^t$ is formulated as follows:

$$b_{i,k}^t = h_i^t \mathbf{W}_Q \mathbf{W}_R^T e_{v_k}^T \quad (13)$$

where $\mathbf{W}_R$ is an MLP with the shape of $d \times d$. The proposed computation simplifies the approach introduced in [6].

Summarizing the aforementioned process, the output of the interaction perception module denoted as follows:

$$v_i^t = \text{softmax}\left(\frac{(h_i^t \mathbf{W}_Q)(\bar{\mathbf{H}}_i^t \mathbf{W}_K)^T + \mathbf{B}_i^t}{\sqrt{d}}\right) \bar{\mathbf{H}}_i^t \mathbf{W}_V \in \mathbb{R}^d, \quad (14)$$

where $\mathbf{B}_i^t = h_i^t \mathbf{W}_Q \mathbf{W}_R^T \bar{\mathbf{E}}_i^t$. Aggregating it with h_i^t yields:

$$\tilde{h}_i^t = h_i^t + v_i^t. \quad (15)$$

Temporal Awareness (TA) module. Subsequently, we input $\tilde{h}_i^t$ into the TA module (a GRU layer [5]) to integrate historical information, thereby mitigating the limitations of POMDP and providing spatiotemporal information for subsequent action generation and state evaluation, we denoted the output as z_i^t .

Group Action Decoder (GAD). In SMAPO, the policy $\pi^\theta(A^t|S^t)$ models the conditional distribution of group action given the joint state S^t . As illustrated in the right of Figure 1, for each agent i in agent group, we output its action a_i^t as:

$$\Delta(a_i^t) = \phi(z_i^t; \mathbf{W}_p), a_i^t \sim \Delta(a_i^t) \quad (16)$$

where $\phi(\cdot; \mathbf{W}_p)$ is an MLP Layer with the shape of $d \times |A|$. For the critic, we calculate the value of agent i as $V(s_i) = \phi(z_i^t; \mathbf{W}_v)$ using a MLP with the shape of $\mathbf{W}_v \in \mathbb{R}^{d \times 1}$.

4.3 Training with Dual Transformation (DT)

Data augmentation strategies have been proven to be important in the learning-to-optimize area [17, 20]. In this study, we develop a Dual Transformation (DT) strategy for training our SMAPO by establishing several key properties of LMAPF.

Given a scene graph G , a set of N agents, a set of start locations ST , and a set of goal locations ED , the LMAPF problem can be formally described as: $U = \langle G, N, ST, ED \rangle$, the set of solutions for problem U is defined as $\text{Solution}(U)$. The DT of LMAPF is defined as:

$$\mathcal{F} : U \rightarrow U',$$

which satisfies:

$$\text{Soutlion}(\mathcal{F}(U)) = \mathcal{F}(\text{Soutlion}(U)).$$

Based on the aforementioned definition, we have demonstrated that symmetric and rotational transformations are Dual Transformations of LMAPF. The proof is provided in Section B.3 of Supplementary Material. Given a Dual Transformation $\mathcal{F}$, we define the dual problem of U to be $\mathcal{F}(U)$.

Within the actor-critic modeling framework previously delineated, by employing the definition of Dual Transformation, we have delineated the respective relationships of optimal policy and optimal value function to $\mathcal{F}(U)$ and U :

$$\mathcal{F}(\pi^*(z)) = \pi^*(\mathcal{F}(z)), \quad (17)$$

$$V^*(z) = V^*(\mathcal{F}(z)), \quad (18)$$

Table 1: Experiment results on 65×65 Maze-like maps (Unseen during training). We set a time limit of 60 seconds per planning step and 8 hours for the total planning process. '-' denotes that the algorithm exceeds the given time limits.

Method	wc4-od30-128		wc7-od30-128		wc2-od45-128		wc5-od50-128		wc2-od55-128		wc3-od65-128	
	TP $\uparrow$	Time (S) $\downarrow$	TP $\uparrow$	Time (S) $\downarrow$	TP $\uparrow$	Time (S) $\downarrow$	TP $\uparrow$	Time (S) $\downarrow$	TP $\uparrow$	Time (S) $\downarrow$	TP $\uparrow$	Time (S) $\downarrow$
RHCR	2.330	11.363	2.240	11.149	2.495	11.273	2.063	11.898	2.438	10.557	2.364	9.875
PRIMAL2	1.477	47.652	1.267	54.350	1.530	49.586	1.046	50.461	1.385	52.742	1.239	72.026
SCRIMP	0.667	265.367	0.647	449.862	0.618	215.718	0.467	466.435	0.588	214.017	0.593	257.936
MAPF-GPT	1.143	43.627	0.484	46.401	1.977	47.220	1.510	42.774	1.990	44.664	1.820	44.079
MATS-LP	1.796	997.469	1.908	818.052	2.107	740.002	1.586	783.472	2.139	733.279	1.947	738.520
FOL	1.797	5.961	1.732	5.680	1.995	5.632	1.527	5.569	1.961	5.539	1.793	5.461
DT-FOL	1.872	6.195	1.777	5.892	2.030	5.858	1.575	5.763	2.021	5.765	1.887	5.875
SMAPO	2.007	9.709	1.929	8.967	2.157	8.898	1.718	9.398	2.129	8.721	2.001	8.712

Method	wc4-od30-512		wc7-od30-512		wc2-od45-512		wc5-od50-512		wc2-od55-512		wc3-od65-512	
	TP $\uparrow$	Time (S) $\downarrow$	TP $\uparrow$	Time (S) $\downarrow$	TP $\uparrow$	Time (S) $\downarrow$	TP $\uparrow$	Time (S) $\downarrow$	TP $\uparrow$	Time (S) $\downarrow$	TP $\uparrow$	Time (S) $\downarrow$
RHCR	-	-	-	-	4.127	9249.148	-	-	3.963	6588.730	3.049	10980.537
PRIMAL2	1.258	358.432	1.243	323.390	1.579	311.271	1.143	318.253	1.549	267.14	1.420	276.320
SCRIMP	-	-	-	-	0.941	24384.112	-	-	0.815	28740.876	0.787	28054.339
MAPF-GPT	0.595	247.830	0.399	251.914	0.770	252.129	0.520	248.216	0.721	248.680	0.721	247.636
MATS-LP	1.834	18081.280	1.584	20162.462	2.340	14674.414	1.076	17612.745	2.250	10568.892	1.721	9457.450
FOL	2.530	21.372	2.430	21.426	3.082	21.385	1.693	23.274	2.959	22.618	2.246	24.315
DT-FOL	2.728	21.886	2.688	22.277	3.445	22.275	1.995	25.330	3.302	21.860	2.556	23.245
SMAPO	3.877	42.440	3.698	42.451	4.411	44.461	2.696	42.674	4.230	44.413	3.491	43.271

By setting $z = z_t^f$, we obtain the characterizing properties of the optimal policy and value heads. Proofs of (17) and (18) are provided in Section B.4 of Supplementary Material.

To achieve high sample efficiency, we derive a decoupled lower bound for the multi-agent joint objective through formal modeling of the Decision Refinement phase. This bound proves structurally equivalent to the Independent Proximal Policy Optimization (IPPO) [41]. For a comprehensive analysis, refer to Section A of Supplementary Material. Leveraging the aforementioned properties and denoted the Dual Transformation set we used as F , we derive the specific policy and value loss for training as follows:

$$L_{policy} = \frac{1}{NT|F|} \sum_{t=1}^T \sum_{k=1}^N \sum_{l=1}^{|F|} \min[r_{t,l}^{\pi_k}(\theta) \hat{A}_{t,l}^k, \text{clip}(r_{t,l}^{\pi_k}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{t,l}^k]. \quad (19)$$

$$L_{value} = \frac{1}{2NT|F|} \sum_{t=1}^T \sum_{k=1}^N \sum_{l=1}^{|F|} (V_{t,l}^{\pi_k}(\theta) - \hat{V}_{t,l}^{\pi_k})^2. \quad (20)$$

in which $r_{t,l}^{\pi_k}(\theta) = \frac{\pi_k^\theta(a_{k,l}^t | f_l(s_k^t))}{\pi_k^{\theta_{old}}(a_{k,l}^t | f_l(s_k^t))}$, $V_{t,l}^{\pi_k}(\theta) = V_k^\theta(f_l(s_k^t))$ and $\hat{V}_{t,l}^{\pi_k} = r_t + \gamma r_{t+1} + \dots + \gamma^{T-t+1} r_{T-1} + \gamma^T V_{T,l}^{\pi_k}$. $\hat{A}_{t,l}^k$ is computed based on $V_{t,l}^{\pi_k}$ using the Generalized Advantage Estimation (GAE) [31]:

$$\hat{A}_{t,l}^k = \sum_{i=0}^{T-t} (\gamma \lambda)^i \delta_{t+i,l}^k, \quad (21)$$

where $\delta_{t+i,l}^k = r_{t+i} + \gamma V_{t+i+1,l}^{\pi_k} - V_{t+i,l}^{\pi_k}$. Here, $V_{t+i,l}^{\pi_k}$ is generated by model during data sampling.

5 Experiments

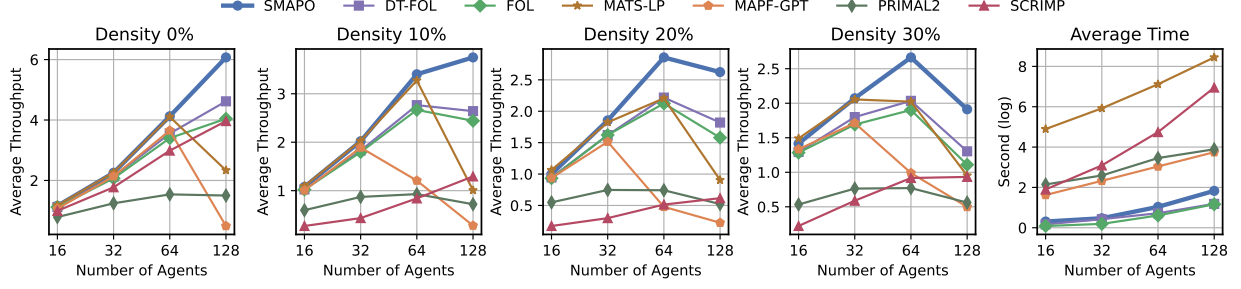
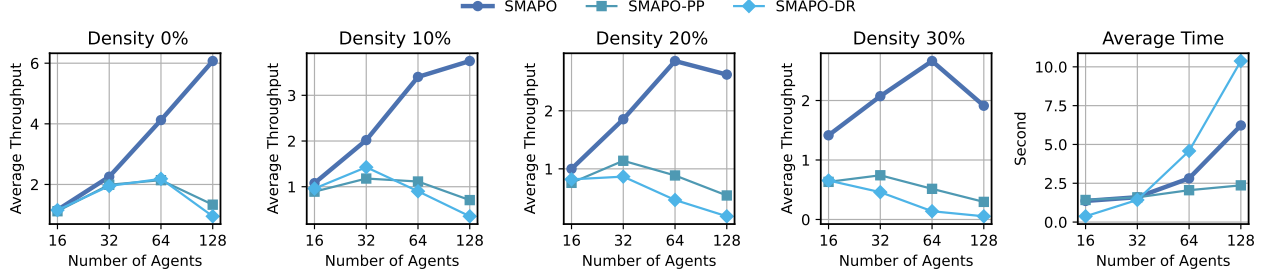
5.1 Experiment Setup

Baselines. For search-based methods, we choose the state-of-the-art RHCR [19]. For learning-based methods, we select SCRIMP [40], PRIMAL2 [7], MAPF-GPT [1], MATS-LP [35] and Follower (FOL) [34], which achieves the state-of-the-art performance against other learning-based baselines. Additionally, we added an extra baseline, DT-Follower (DT-FOL), which integrates DT into Follower's training. Notably, all learning-based methods (including our SMAPO) were trained once and evaluated in zero-shot settings across maps with varying topologies and scales. The concrete settings of these baselines can be found in Appendix A.1.

Training details. During both the training and testing phases, the episode length is set to 512. Each agent's visible range is 11×11 which is the same as in PRIMAL2, SCRIMP, MATS-LP, MAPF-GPT, DT-FOL and FOL. A complete training of SMAPO includes 60 million environmental sampling steps in approximately 2.5 hours. All experiments are run on an Intel(R) Xeon(R) 6348 CPU, with an NVIDIA GeForce RTX 3090 GPU and 504G RAM. Refer to Appendix A.1 for more details.

5.2 Performance Comparisons

To highlight the performance and generalization capabilities of our methods, we conduct experiments across four distinct map types: the 65×65 Maze-like maps used for training FOL, PRIMAL2, MAPF-GPT, MATS-LP and our SMAPO; the 20×20 random maps from the PICO environment, which were used for training SCRIMP; the city map and random map from the renowned MovingAI benchmark [37], compressed to a size of 64×64 . These four map types exhibit vastly different topological structures, particularly the city maps derived from real-world data in the MovingAI benchmark. Each data point is the average of results from 10 seeds. Due to space

Figure 2: Experiment results on 20×20 Pico-Random mapsFigure 3: The evaluation of PP phase and DR phase on 20×20 Pico-Random maps.

limitations, only the results on maze-type maps and pico-random maps are presented in the main paper, with additional comparisons on Random and City maps provided in Appendix A.3.

Results on Maze-like map. Table 1 reports the average *throughput* (TP) and total planning time of various algorithms on maze-type maps with different map complexity and number of agents. The best results for each comparison group are marked in bold. Note that in the nomenclature of maze-type maps, ‘wc’ and ‘od’ represent world corridor and obstacle density [7], respectively, with their combination determining the overall complexity of maze maps.

It can be observed that the SMAPO model significantly outperforms other learning-based approaches across maze-type maps with varying topologies and scales. This advantage becomes more pronounced as the number of agents increases, indicating that our methods can improve data utilization and further enhance the agents’ ability to cooperate and reach their goals at a lower cost in complex environments. For the other learning-based method, MATS-LP achieves a performance comparable to SMAPO in small-scale scenarios through MCTS integration, although it incurs approximately $100\times$ computational cost. However, the performance of MATS-LP significantly declines as agent populations increase, primarily due to the exponential growth of MCTS search space.

In comparison with the search-based method RHCR, SMAPO demonstrates superior performance and computational efficiency in large-scale multi-agent scenarios. Our analysis reveals that RHCR’s computational time increases rapidly with both agent population growth and environmental complexity. Notably, algorithmic efficiency of RHCR exhibits significant sensitivity to world corridor (wc), failing to produce solutions within planning time constraints when wc exceeds 3. This phenomenon stems from increased maze complexity with larger wc values, characterized by elongated corridors. For search-based methods, this complexity results in a significant increase in invalid search paths, necessitating substantial

additional computational time. In contrast, SMAPO maintains consistent computational efficiency in varying environmental complexities while achieving remarkable performance metrics.

Results on Pico-Random map. To further investigate the generalization of learning-based methods, we conducted additional evaluations on Pico-Random maps (20×20) as shown in Figure 2. We observed that on obstacle-free Pico-Random maps, SMAPO, FOL, DT-FOL, and SCRIMP demonstrate scalable throughput performance. However, at 10% obstacle density, only SMAPO sustains throughput improvement and peak performance when scaling from 64 to 128 agents. This demonstrates SMAPO’s superior robustness in managing complex coordination during agent population growth in challenging environments, outperforming baselines.

The rightmost part of Figure 2 illustrates the changes in the planning time (log scale) as the number of agents increases. The result demonstrates that MATS-LP exhibits relatively low algorithmic efficiency, rapidly decreasing with the growth of agent numbers due to its MCTS implementation. SCRIMP shows similar performance degradation, caused by additional state value evaluations proportional to the number of agents in its decision-making process. In comparison, both SMAPO and FOL maintain consistently high operational efficiency while SMAPO significantly outperforms FOL, further validating the effectiveness of our SMAPO.

In addition, a notable observation is that neural-only methods such as SCRIMP, PRIMAL2, MAPF-GPT, exhibit weaker generalization and environmental adaptation compared to search-integrated learning methods, including our SMAPO and FOL. This performance gap occurs because LMATP’s complex multi-task nature often cause RL/IL algorithms to overfit training data, limiting their ability to learn generalizable policies. For MATS-LP, the inefficiency of MCTS in multi-agent environments significantly limits its scalability in large-scale agent scenarios. Consequently, we next focus

Table 2: Performance of different ablated baselines on City map from MovingAI.

Method	SHA-128	LON-128	BER-128	SYD-128	MOW-128	DEN-128	MIL-128	NYC-128	BOS-128	Average Time (S)
SMAPO-LB	3.312	4.125	2.889	3.121	3.364	2.258	2.462	2.695	2.250	4.445
SMAPO-PI	3.828	4.757	3.539	3.485	3.837	2.646	3.367	2.946	3.004	5.925
SMAPO-Pe	3.693	4.471	3.361	3.493	3.579	2.659	3.118	2.944	2.346	7.808
SMAPO-IP	3.785	4.683	3.436	3.453	3.802	2.654	3.340	2.953	2.956	6.769
SMAPO-RVE	3.898	4.785	3.634	3.531	3.933	2.713	3.425	3.010	3.069	6.856
SMAPO-ROPE	3.896	4.841	3.603	3.554	3.950	2.708	3.433	3.021	3.096	7.275
SMAPO-TA	3.890	4.800	3.637	3.552	3.929	2.713	3.425	3.023	3.078	7.881
SMAPO-DT	3.865	4.748	3.602	3.531	3.864	2.704	3.394	2.977	3.054	6.976
SMAPO	3.912	4.853	3.659	3.579	3.956	2.716	3.443	3.027	3.145	6.998
Method	SHA-512	LON-512	BER-512	SYD-512	MOW-512	DEN-512	MIL-512	NYC-512	BOS-512	Average Time (S)
SMAPO-LB	7.141	9.169	4.124	5.706	6.918	3.170	5.078	4.299	3.698	20.446
SMAPO-PI	9.738	10.541	5.352	7.711	9.305	6.782	7.713	6.746	6.648	25.715
SMAPO-Pe	7.891	9.564	4.350	6.451	7.994	4.948	6.696	5.013	5.141	47.348
SMAPO-IP	9.768	10.441	5.469	7.983	9.146	6.648	7.835	7.064	6.752	46.401
SMAPO-RVE	10.104	11.018	5.711	8.136	9.989	7.640	8.412	7.273	7.347	53.034
SMAPO-ROPE	9.657	11.021	5.854	8.076	9.842	7.552	8.268	7.237	7.518	57.726
SMAPO-TA	9.688	11.006	5.634	8.285	10.056	7.660	8.529	7.429	7.489	56.528
SMAPO-DT	9.509	10.524	5.709	7.727	9.288	6.949	7.751	6.606	6.626	47.046
SMAPO	10.425	11.608	6.230	8.795	10.528	7.838	8.840	7.629	7.764	47.508

on the comparative evaluation of the efficient search-integrated learning methods, namely our proposed models and FOL.

5.3 In-depth Analysis of SMAPO

In this section, we conduct an evaluation of the performance impacts associated with various design components of SMAPO. The assessment proceeds in a hierarchical manner: initially, we evaluate the overall effectiveness of the Primordial Planning phase and the Decision Refinement phase. Subsequently, we perform a detailed analysis of each specific design element within these two phases, as well as the Dual Transformation (DT).

Impact of PP phase and DR phase. To evaluate the impact of the Primordial Planning (PP) and Decision Refinement (DR) phases, we design two ablations 1) **SMAPO-PP**, which trains a model solely based on the DR phase by removing the PP phase from SMAPO; and 2) **SMAPO-DR**, which makes decisions directly using the planning results from the PP phase. As shown in Figure 3, both the absence of PP phase and DR phase significantly degrade the performance on Pico-Random map. Notably, SMAPO-DR exhibits longer runtime than SMAPO ($\Delta = 64.73\%$ on average). We attribute this to suboptimal planning in PP phase: 1) The retained conflicts from PP phase destabilize the PeCC; 2) Such instability reduces heuristic reliability, impeding search efficiency and prolonging computation time. This proves that combining PP and DR phases ensures both superior performance and efficiency.

Planning Algorithm Design in the PP phase. As described in Section 4.1, we developed LB-A* to rapidly generate a congestion-aware initial path set during the PP phase. To assess the performance advantages of LB-A* and its individual components, we designed three ablation studies: 1) **SMAPO-LB**, which replaces LB-A* with the classic A* [11]; 2) **SMAPO-PI**, which removes PICC from the cost calculation; and 3) **SMAPO-Pe**, which excludes PeCC from the cost computation. We report the evaluation results on City map from MovingAI in Table 2. The table demonstrates that SMAPO exhibits significantly superior performance compared to SMAPO-PI,

SMAPO-Pe, and SMAPO-LB. Notably, SMAPO-LB shows the weakest performance in all tested scenarios. These findings reveal two key observations: 1) Both PICC and PeCC substantially influence the performance of LB-A*; 2) Although removing PICC or PeCC degrades the quality of LB-A*'s path sets, they still outperform classical A* algorithms. This performance disparity stems from the implicit collaboration embedded in PICC and PeCC designs, which facilitates multi-agent coordination in LMAPF systems to secure balanced traffic load.

Network Architecture Design in the DR phase. As detailed in Section 4.2, we proposed a neural network to optimize the initial path configurations during the DR phase. Here, we investigate on what extent these designs influence SMAPO's learning effectiveness. Concretely, for the Interaction Perception module (IP), we introduce one ablation SMAPO-IP: cancel the information sharing between the nearby agents by replacing the IP module by an MLP layer. For the Relative Vector Embedding(RVE) used in IP module, we introduce two ablations 1) SMAPO-RVE: remove the RVE from SMAPO; 2) SMAPO-ROPE, replace RVE by Two-Dimensional Rotary Position Embedding (2D-ROPE) [13]. For the Temporal Awareness (TA) module, we introduce one ablation SMAPO-TA: replacing the TA module with an MLP layer to cancel historical data utilization.

The results in Table 2 underscores the importance of these special designs: (a) Absence of the IP module hinders SMAPO's ability to learn effective collaborative decision policy. (b) RVE plays a crucial role in embedding complex cooperative patterns among agents in LMAPF tasks. Both the removal of RVE and its replacement with 2D-ROPE lead to performance degradation. (c) Elimination of the TA module disrupts historical information utilization in SMAPO, suppressing potential reflective behaviors, which is particularly detrimental under POMDP assumptions.

Analysis of DT's effectiveness To evaluate DT's effectiveness, we compared SMAPO with SMAPO-DT (SMAPO without DT) and FOL with DT-FOL (FOL with DT). Results from Tables 1-2 and Figure 2 reveal that: 1) DT enhances both SMAPO and FOL training, demonstrating its general applicability in LMAPF tasks; 2) DT's

Table 3: The evaluation of key hyperparameters in PP phase.

(α, β, λ)	wc4-od30-512	wc7-od30-512	wc2-od45-512	Total time (s)
(1, 0.3, 0.8)	3.713	3.590	4.331	114.174
(1, 0.5, 0.8)	3.828	3.589	4.307	115.892
(1, 0.7, 0.8)	3.809	3.658	4.384	122.021
(2, 0.3, 0.8)	3.780	3.675	4.421	123.581
(2, 0.5, 0.8)	3.877	3.698	4.411	124.192
(2, 0.5, 0.6)	3.838	3.689	4.413	123.132
(2, 0.5, 1.0)	3.868	3.692	4.481	125.050
(2, 0.7, 0.8)	3.872	3.684	4.502	126.435
(3, 0.3, 0.8)	3.923	3.658	4.467	125.798
(3, 0.5, 0.8)	3.890	3.720	4.503	126.614
(3, 0.7, 0.8)	3.919	3.738	4.511	127.673
(20, 0.5, 0.8)	4.086	3.825	4.695	149.501

Table 4: The evaluation of different Reward Scale.

Reward	wc4-od30	wc7-od30	wc2-od45	pico-od0	pico-od10	pico-od20
0.002	1.708	1.874	2.291	3.963	2.268	1.551
0.01	3.877	3.698	4.411	6.071	3.750	2.623
0.05	3.820	3.652	4.372	5.875	3.568	2.398

impact is more significant in scenarios with a larger number of agents, indicating its enhanced utility in helping agents adapt to highly congested environments.

5.4 Analysis Of Hyperparameters

In this section, we evaluate the impact of key hyperparameters in SMAPO, including those in LB-A* and RL.

Hyperparameters in LB-A*. As introduced in Section 4.1, the LB-A* involves four key parameters: 1) α adjusts conflict timing costs; 2) β controls baseline proximity costs between agents within the time window; 3) λ regulates the decay rate of these costs; 4) γ determines the influence of past PeCC values on current planning. For γ , we set it as $0.5^{\frac{1}{\text{map}_w + \text{map}_h}}$, where map_h and map_w denote the map's height and width, respectively.

The experimental findings in Table 5.3 demonstrate that SMAPO maintains stable performance when $\alpha \in [1, 3]$, $\beta \in [0.3, 0.7]$, $\lambda \in [0.6, 1.0]$. Parametric analysis reveals: 1) Higher α values consistently enhance path quality but increase search duration. 2) SMAPO maintains robustness across different β and λ values. Our SMAPO chooses $\alpha = 2$ and $\beta = 0.5$ to achieve balance between solution quality and computational efficiency.

Hyperparameters in RL. As detailed in Section 4.2, we retained most hyperparameters from IPPPO [41] owing to structural similarity of our loss function. The key hyperparameter under consideration is the reward scale, which balances signal discriminability and value function stability during training. The results in Table 4 support the initial hypothesis: excessive reward destabilizes critic predictions (hurting performance), while too insufficient reward weakens policy gradients, reducing sample efficiency.

5.5 Noisy and Dynamic Environment Validation

To validate the robustness of SMAPO, we conducted real-world simulations evaluating three key failure modes: 1) PeCC update

Table 5: The Validation of Noisy and Dynamic Environment.

(p_1, p_2, p_3)	wc4-od30-512	wc7-od30-512	wc2-od45-512
(0, 0, 0)	3.877	3.698	4.411
(0, 0, 0.05)	3.856	3.664	4.369
(0, 0, 0.1)	3.806	3.628	4.358
(0, 0.2, 0)	3.862	3.719	4.405
(0.2, 0, 0)	3.932	3.708	4.495
(0.2, 0.2, 0)	3.938	3.705	4.488

latency with probability p_1 (delay the update to the next timestep); 2) PeCC update packet loss with probability p_2 (the update never arises); 3) PP $\rightarrow$ DR phase communication delays with probability p_3 (DR phase can only access the planned path from the previous timestep). The results in Table 5 reveal SMAPO's robustness against real-world operational challenges, including perceptual latency, communication delays, and packet loss: 1) Even under 20% probability of delayed PeCC updates and packet loss, SMAPO maintained stable performance. 2) SMAPO exhibited consistent resilience at 0.1 probability of PP-DR phase information latency.

6 Conclusion

In this paper, we propose SMAPO, a search-integrated learning framework for rapid generation and optimization of load-balanced path sets. SMAPO comprises two tightly-coupled phases: 1) Primal Planning phase employing a novel load-balancing A* algorithm that integrates planning and perceived congestion costs to enhance solution quality through spatiotemporal load distribution and conflict mitigation; 2) Decision Refinement phase formulate collaborative path set optimization as the primary objective and employ multi-agent reinforcement learning to develop an Encoder-Decoder-based cooperative policy. Additionally, we utilize dual transformations to augment trajectory samples during online learning, boosting sample utilization and overall learning stability. Extensive evaluations on LMAPF benchmarks demonstrate SMAPO's consistent superiority over state-of-the-art learning methods and its superior performance-efficiency balance compared to search-based approaches. Future work involves enriching agent action diversity and extending SMAPO to dynamic obstacle environments.

7 Acknowledgement

This work was supported in part by Guangdong Provincial Natural Science Foundation for Outstanding Youth Team Project (Grant No. 2024B1515040010), in part by National Natural Science Foundation of China (Grants No. 62206212 & 62276100), in part by Guangzhou Science and Technology Elite Talent Leading Program for Basic and Applied Basic Research (Grant No. SL2024A04J01361), and in part by the Fundamental Research Funds for the Central Universities (Grant No. ZYTS25297).

References

- [1] Anton Andreychuk, Konstantin Yakovlev, Aleksandr Panov, and Alexey Skrynnik. 2025. MAPF-GPT: Imitation Learning for Multi-Agent Pathfinding at Scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39, 23126–23134.
- [2] Daniel S Bernstein, Robert Givan, Neil Immerman, and Shlomo Zilberstein. 2002. The Complexity of Decentralized Control of Markov Decision Processes. *Mathematics of Operations Research* 27, 4 (2002), 819–840.
- [3] Jiacheng Chen, Zeyuan Ma, Hongshu Guo, Yining Ma, Jie Zhang, and Yue-Jiao Gong. 2024. Symbol: Generating Flexible Black-Box Optimizers through Symbolic Equation Learning. In *The Twelfth International Conference on Learning Representations*.
- [4] Hao-Tien Lewis Chiang, Jasmine Hsu, Marek Fiser, Lydia Tapia, and Aleksandra Faust. 2019. RL-RRT: Kinodynamic motion planning via learning reachability estimators from RL policies. *IEEE Robotics and Automation Letters* 4, 4 (2019), 4298–4305.
- [5] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. 2014. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555* (2014).
- [6] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc Le, and Ruslan Salakhutdinov. 2019. Transformer-XL: Attentive Language Models beyond a Fixed-Length Context. In *Annual Meeting of the Association for Computational Linguistics*. 2978–2988.
- [7] Mehul Damani, Zhiyao Luo, Emerson Wenzel, and Guillaume Sartoretti. 2021. PRIMAL₂: Pathfinding Via Reinforcement and Imitation Multi-Agent Learning - Lifelong. *IEEE Robotics and Automation Letters* 6, 2 (2021), 2666–2673.
- [8] Zibin Dong, Jianye Hao, Yifu Yuan, Fei Ni, Yitian Wang, Pengyi Li, and Yan Zheng. 2024. DiffuserLite: Towards Real-time Diffusion Planning. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 122556–122583.
- [9] Florian Grenouilleau, Willem-Jan Van Hoeve, and John N Hooker. 2019. A Multi-Label A* Algorithm for Multi-Agent Pathfinding. In *Proceedings of the international conference on automated planning and scheduling*, Vol. 29, 181–185.
- [10] Hongshu Guo, Zeyuan Ma, Jiacheng Chen, Yining Ma, Zhiguang Cao, Xinglin Zhang, and Yue-Jiao Gong. 2025. ConfigX: Modular Configuration for Evolutionary Algorithms via Multitask Reinforcement Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [11] Peter E Hart, Nils J Nilsson, and Bertram Raphael. 1968. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics* 4, 2 (1968), 100–107.
- [12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 770–778.
- [13] Byeongho Heo, Song Park, Dongyoon Han, and Sangdoo Yun. 2024. Rotary Position Embedding for Vision Transformer. In *European Conference on Computer Vision*. Springer, 289–305.
- [14] Florence Ho, Artur Goncalves, Ana Salta, Marc Cavazza, Ruben Geraldes, and Helmut Prendinger. 2019. Multi-Agent Path Finding for UAV Traffic Management: Robotics Track. (2019).
- [15] Michael Janner, Yilun Du, Joshua Tenenbaum, and Sergey Levine. 2022. Planning with Diffusion for Flexible Behavior Synthesis. In *International Conference on Machine Learning*. PMLR, 9902–9915.
- [16] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The Efficient Transformer. *arXiv preprint arXiv:2001.04451* (2020).
- [17] Yeong-Dae Kwon, Jinho Choo, Byoungjip Kim, Iljoo Yoon, Youngjune Gwon, and Seungjai Min. 2020. POMO: Policy optimization with multiple optima for reinforcement learning. *Advances in Neural Information Processing Systems* 33 (2020), 21188–21198.
- [18] Jiaoyang Li, Wheeler Ruml, and Sven Koenig. 2021. EECBS: A Bounded-Suboptimal Search for Multi-Agent Path Finding. In *Proceedings of the AAAI conference on Artificial Intelligence*, Vol. 35, 12353–12362.
- [19] Jiaoyang Li, Andrew Tinka, Scott Kiesel, Joseph W Durham, TK Satish Kumar, and Sven Koenig. 2021. Lifelong Multi-Agent Path Finding in Large-Scale Warehouses. In *Proceedings of the 35th AAAI Conference on Artificial Intelligence*. 11272–11281.
- [20] Qi Li, Zhiguang Cao, Yining Ma, Yaoxin Wu, and Yue-Jiao Gong. 2025. Diversity Optimization for Travelling Salesman Problem via Deep Reinforcement Learning. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '25)*. 683–694.
- [21] Qingbiao Li, Weizhe Lin, Zhe Liu, and Amanda Prorok. 2021. Message-Aware Graph Attention Networks for Large-Scale Multi-Robot Path Planning. *IEEE Robotics and Automation Letters* 6, 3 (2021), 5533–5540.
- [22] Wenhao Li, Hongjun Chen, Bo Jin, Wenzhe Tan, Hong Zha, and Xiangfeng Wang. 2022. Multi-Agent Path Finding with Prioritized Communication Learning. In *International Conference on Robotics and Automation*. 10695–10701.
- [23] Hang Ma, Daniel Harabor, Peter J Stuckey, Jiaoyang Li, and Sven Koenig. 2019. Searching with Consistent Prioritization for Multi-Agent Path Finding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33, 7643–7650.
- [24] Hang Ma, Jiaoyang Li, TK Kumar, and Sven Koenig. 2017. Lifelong Multi-Agent Path Finding for Online Pickup and Delivery Tasks. In *Proceedings of the 16th Conference on Autonomous Agents and Multiagent Systems*. 837–843.
- [25] Zeyuan Ma, Hongshu Guo, Jiacheng Chen, Zhenrui Li, Guojun Peng, Yue-Jiao Gong, Yining Ma, and Zhiguang Cao. 2024. MetaBox: a benchmark platform for meta-black-box optimization with reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2024).
- [26] Ziyuan Ma, Yudong Luo, and Hang Ma. 2021. Distributed Heuristic Multi-Agent Path Finding with Communication. In *2021 IEEE International Conference on Robotics and Automation*. IEEE, 8699–8705.
- [27] Robert Morris, Corina S Pasareanu, Kasper Luckow, Waqar Malik, Hang Ma, TK Satish Kumar, and Sven Koenig. 2016. Planning, Scheduling and Monitoring for Airport Surface Operations. In *Workshops at the Thirtieth AAAI Conference on Artificial Intelligence*.
- [28] Ayano Okoso, Keisuke Otaki, and Tomoki Nishi. 2019. Multi-Agent Path Finding with Priority for Cooperative Automated Valet Parking. In *2019 IEEE Intelligent Transportation Systems Conference*. IEEE, 2135–2140.
- [29] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.
- [30] Guillaume Sartoretti, Justin Kerr, Yunfei Shi, Glenn Wagner, TK Satish Kumar, Sven Koenig, and Howie Choset. 2019. PRIMAL: Pathfinding via Reinforcement and Imitation Multi-Agent Learning. *IEEE Robotics and Automation Letters* 4, 3 (2019), 2378–2385.
- [31] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. 2015. High-Dimensional Continuous Control Using Generalized Advantage Estimation. *arXiv preprint arXiv:1506.02438* (2015).
- [32] Guni Sharon, Roni Stern, Ariel Felner, and Nathan R. Sturtevant. 2015. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence* 219 (2015), 40–66.
- [33] David Silver. 2005. Cooperative Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 1, 117–122.
- [34] Alexey Skrynnik, Anton Andreychuk, Maria Nesterova, Konstantin Yakovlev, and Aleksandr Panov. 2024. Learn to Follow: Decentralized Lifelong Multi-agent Pathfinding via Planning and Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38, 17541–17549.
- [35] Alexey Skrynnik, Anton Andreychuk, Konstantin Yakovlev, and Aleksandr Panov. 2024. Decentralized Monte Carlo Tree Search for Partially Observable Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38, 17531–17540.
- [36] Alexey Skrynnik, Anton Andreychuk, Konstantin Yakovlev, and Aleksandr I Panov. 2023. When to Switch: Planning and Learning for Partially Observable Multi-Agent Pathfinding. *IEEE Transactions on Neural Networks and Learning Systems* (2023).
- [37] Roni Stern, Nathan R Sturtevant, Ariel Felner, Sven Koenig, Hang Ma, Thayne T Walker, Jiaoyang Li, Dor Atzmon, Liron Cohen, TK Satish Kumar, et al. 2019. Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks. In *Proceedings of the 12th Annual Symposium on Combinatorial Search*. 151–158.
- [38] Rishi Veerapaneni, Muhammad Suhail Saleem, Jiaoyang Li, and Maxim Likhachev. 2025. Windowed MAPF with Completeness Guarantees. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [39] Thayne T Walker, Nathan R Sturtevant, Ariel Felner, Han Zhang, Jiaoyang Li, and TK Satish Kumar. 2021. Conflict-Based Increasing Cost Search. In *Proceedings of the International Conference on Automated Planning and Scheduling*, Vol. 31, 385–395.
- [40] Yutong Wang, Bairan Xiang, Shinan Huang, and Guillaume Sartoretti. 2023. SCRIMP: Scalable Communication for Reinforcement-and Imitation-Learning-Based Multi-Agent Pathfinding. In *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*. 2598–2600.
- [41] Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. 2022. The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games. *Advances in Neural Information Processing Systems* 35 (2022), 24611–24624.
- [42] Shuyang Zhang, Jiaoyang Li, Taoan Huang, Sven Koenig, and Bistra Dilkina. 2022. Learning a priority ordering for prioritized planning in multi-agent path finding. In *Proceedings of the International Symposium on Combinatorial Search*, Vol. 15, 208–216.

Algorithm 1 Load-Balanced A*

Input: neighbors of each cell $Neighbor$, heuristic table H , PeCC table $\mathcal{W}$, number of agents N , start location list ST , goal location list ED , Real-Time Perception $\odot$;

Output: A path set $\mathbb{S}$.

```

1: Initialize  $open \leftarrow \emptyset$ ;  $\mathbb{S} \leftarrow \emptyset$ ;
2: for  $k = 1, 2, \dots, N$  do
3:    $\mathcal{W}_k[c] \leftarrow \gamma \mathcal{W}_k[c] + 1$  for  $c$  in  $\mathcal{O}_k$ ;
4:    $R.pos \leftarrow S_k, R.t \leftarrow 0, R.g \leftarrow 0, R.h \leftarrow H_k[S_k], R.parent \leftarrow NULL$ ;
5:    $open.push(R)$ ; Initialize  $Node$ ;
6:   while  $open$  is not empty do
7:      $p \leftarrow open.pop()$ ;
8:     if  $p.pos = G_k$  then break;
9:     else if  $Node[R.pos].g < p.g$  then continue;
10:    foreach  $n \in Neighbor[p.pos]$  do
11:      update  $cost$  using eq. 1
12:      if  $cost + p.g < Node[n.pos].g$  then
13:         $c.pos \leftarrow n.pos, c.t \leftarrow p.t + 1, c.g \leftarrow cost + p.g$ ;
14:         $c.h \leftarrow H_k[n.pos], c.parent \leftarrow p.pos$ ;
15:         $open.push(c), Node[n.pos] \leftarrow c$ ;
16:    end while
17:    use  $Node$  to obtain  $path_k$ ; add  $path_k$  to  $\mathbb{S}$ ;
18: end for
19: return  $\mathbb{S}$ 

```

A Experiment

A.1 Baseline Settings

To ensure a fair comparison, we trained the Follower (FOL) in the same environment with identical sampling steps (60 million) while maintaining the hyperparameter settings consistent with those in the original Follower paper [34]. In the experiments, MAPF-GPT, MATS-LP, SCRIMP, and PRIMAL2 directly utilized the model parameters and configurations from their open-source implementations. Specifically, MAPF-GPT employed a version with 6 million parameters, while MATS-LP was constrained to 250 expansions. Additionally, it is noteworthy that in our experiments, our method adopted the movement rules of PRIMAL2: when multiple agents attempt to move to the same cell, only one agent succeeds, and the remaining agents remain stationary. In contrast, SCRIMP has its own mechanism for handling such scenarios, which we left unmodified.

RHCR involves four key configuration parameters: the planning algorithm, simulation window, planning window, and planning time limit. For the planning algorithm, we followed the original RHCR implementation by using PBS, as it is the best-performing option. Based on the experimental results of different parameter combinations in the original paper, we set the simulation window and planning window to 5 and 20, respectively. For the planning time limit, we configured a single-step planning time limit of 60 seconds in our experiments. However, due to the simulation window being set to 5, the corresponding RHCR planning time limit per planning cycle was 300 seconds. The RHCR implementation used in our experiments also adhered to these parameter settings.

A.2 Map Visualization

Figure 4 showcases examples of the maps utilized for testing. Based on the map visualizations, we can intuitively observe that different

types of maps exhibit distinct topological structures. The names of the maps presented correspond to those in our repository.

We selected representative maps from various scenarios, including maze maps with wall complexities of wc7-od30 and wc3-od65 (as described in the main text, 'wc' and 'od' denote wall complexity and obstacle density, respectively), pico-random maps with obstacle densities of 10%, 20%, and 30%, urban maps from the MovingAI dataset, and random maps from the same dataset with obstacle densities of 15%, 20%, and 25%.

A.3 Supplementary Experimental Results

Additional Evaluation In the main text, we have presented and thoroughly analyzed the experimental results on maze-type maps and pico-random maps. A key finding is that neural-only methods, such as SCRIMP, PRIMAL2, and MAPF-GPT, exhibit weaker generalization and environmental adaptability compared to search-integrated learning methods, including our SMAPO, Follower (FOL) and DT-Follower (DT-FOL). Although MATS-LP also falls under the category of search-integrated learning methods, its scalability in large-scale multi-agent scenarios is significantly limited by the inefficiency of Monte Carlo Tree Search (MCTS) in such environments. Therefore, here we focus on a comparative evaluation of efficient search-integrated learning methods, specifically our proposed SMAPO, FOL and DT-FOL. We present the results of these methods on City maps and Random maps from the MovingAI Benchmark.

Figure 5 shows two key observations: 1) SMAPO maintains a clear performance advantage on both Random maps and City maps; 2) On Random maps, SMAPO's performance gain over FOL increases from 17.4% to 33.19% when the obstacle density rises from 10% to 25%, tested with 512 agents. These results support our main conclusion: SMAPO consistently performs the best. Its advantage grows as the number of agents increases and the map topology becomes more complex.

B Algorithm Pseudocode

The LB-A* method operates through sequential agent path planning following a predetermined sequence (random ordering in this implementation). We initialize an empty path set $\mathbb{S}$ progressively populating it with three-dimensional trajectories (position and timestep) as each agent's path planned. During agent k 's planning phase: 1) Update the PeCC table $\mathcal{W}_k$ using Real-Time Perception $\odot_k$. 2) Execute forward search analogous to classical A*, employing the cost function in Eq. 1 (Section 4.1) for stepwise cost evaluation.

C Scalability In Prioritized Scenario

SMAPO specifically addresses the MAPF problem with equal-priority agents - a configuration that constitutes both the academic mainstream (as studies compared in Table 1) and the more challenging scenario in coordination complexity. This fundamental setting requires agents to negotiate paths in fully concurrent decision space without priority arbitration. Nonetheless, the prioritized scenario indeed exist in practical applications. Our SMAPO can support priority extension by intuitive adjustments: 1) In the PP phase, priority-based agent sequencing could be implemented, modifying PeCC's bidirectional constraints to unidirectional ones (where

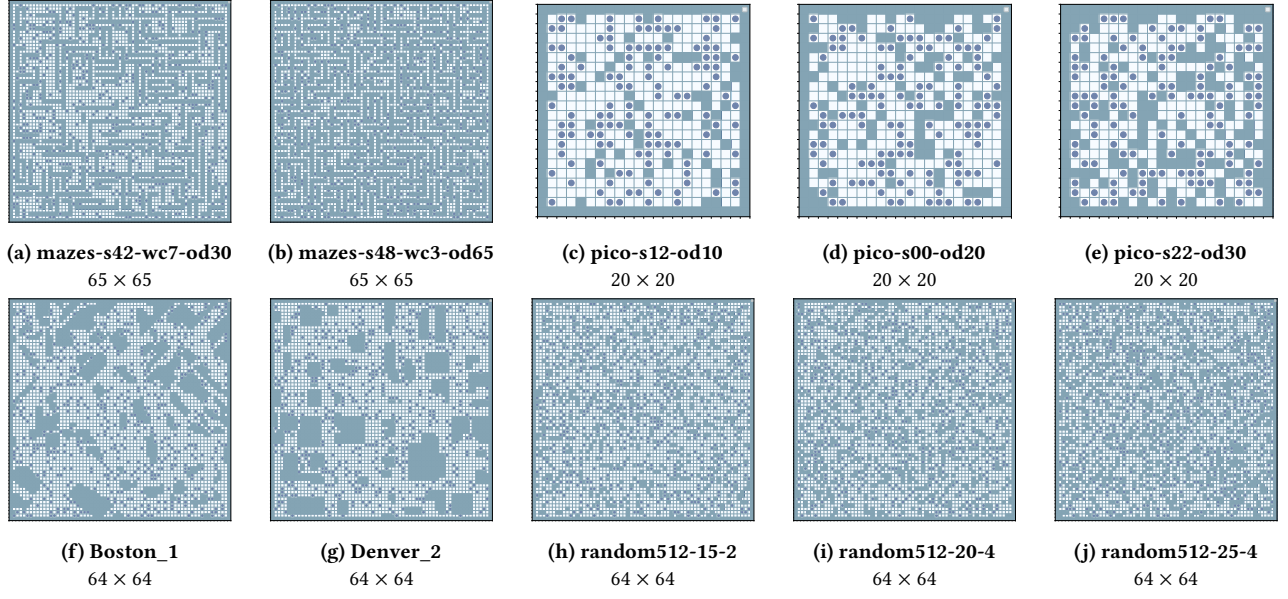


Figure 4: Visualized examples of the maps with a maximum number of agents used for empirical evaluation.

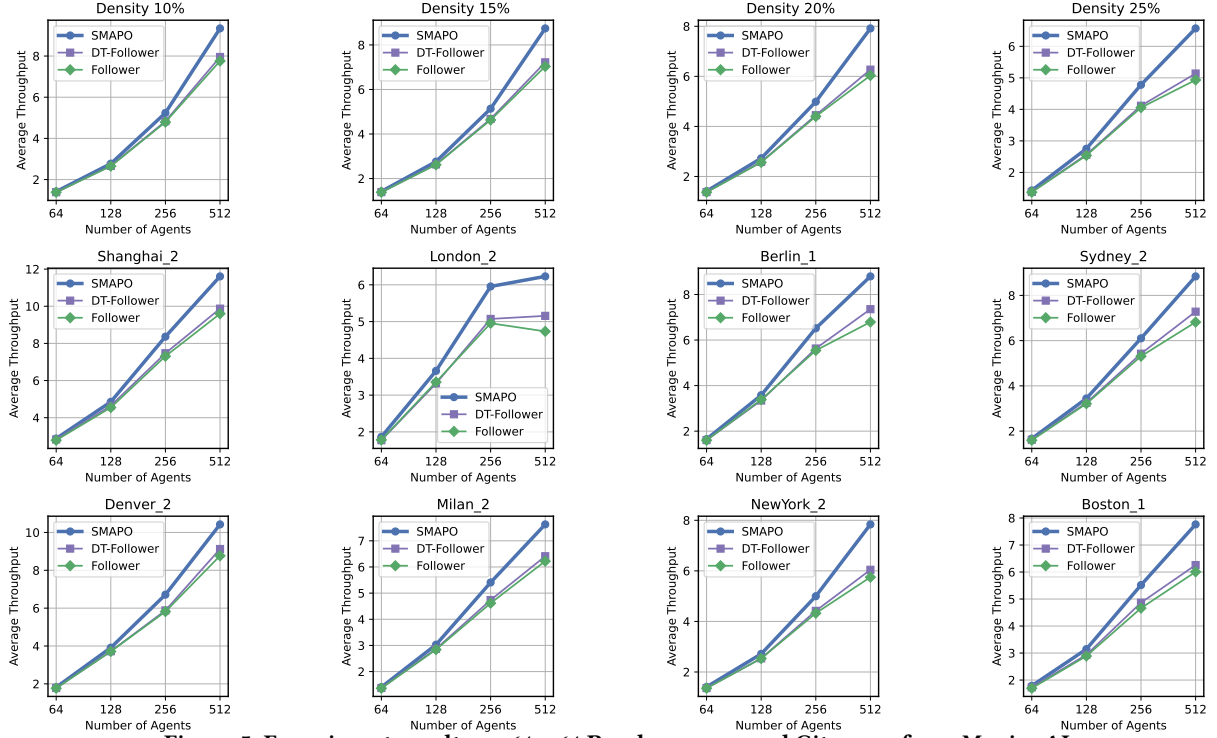


Figure 5: Experiment results on 64×64 Random maps and City map from MovingAI.

higher-priority agents influence lower-priority ones, but not vice versa). 2) For the DR phase, RVE could be enhanced to incorporate priority information, enabling agents to perceive others' priority

levels. 3) During training, priority information could be integrated into the neural network architecture with adjusted reward functions for different priority levels.